Project 1: DBMS Performance Evaluation

https://github.com/hyj0824/cs213-proj-1

Summary

Q1. What are the unique advantages of a DBMS compared with data operations in files?

The advantage is correctness and guarantees under concurrency, not raw speed. In single-threaded mode, the project's file "database" scans the full data in 5.7 s against 6.8 s for PostgreSQL and 11.7 s for openGauss, and it answers 2,000 indexed lookups in 25 ms against 50 and 82 ms.

Once the work is parallel, the databases win: they keep the result correct under concurrent writers, and they supply rollback, constraints, durable commits, and maintained indexes.

Q2. Which DBMS is better? PostgreSQL or openGauss, and by which standard?

PostgreSQL, on almost every standard the project tests: 1.7x on one core, 1.5x at four workers, 1.6x on indexed lookups.

openGauss's USTORE keeps the smallest relation after a growing update, but it has higher update latency and stores undo outside the relation size. That is a space trade-off.

1 Environment

The project has one Java client that tests all three backends through a single interface. The data sits on a host bind mount shared by the containers, which reduces the latency of Docker's `overlay2fs`.

Item	Value
CPU	Intel i5-12400F, 6 cores / 12 threads; 4 server cores, 2 client cores
Memory	16 GiB host; 4 GB `shared_buffers` per engine
Storage	NVMe, ext4; data on a host bind mount
OS	Ubuntu 22.04.5, kernel 6.8.0-138
JVM	OpenJDK 17.0.20.1
Engines	PostgreSQL 18.6; openGauss 7.0.0, both in Docker
Drivers	pgJDBC 42.7.4; openGauss JDBC 5.0.0

Table 1: Hardware and software. The full configuration is in Appendix B.

2 Design

The report answers two questions: what a DBMS adds beyond file I/O, and which of the two engines is better. The same four operations are implemented three times, separately for PostgreSQL, openGauss, and a Java file backend.

2.1 Data and equivalence

The data is the IMDb non-commercial dump with 12,841,585 titles and 2,000,000 person names. The large table is eight copies of the title base with disjoint id ranges, which turns 12.8M rows into 102.7M without changing the title distribution. The sizes tested are 12.8M, 25.7M, 51.4M, and 102.7M rows.

PostgreSQL exports the TSV that the file backend reads, so all three backends see the same rows. A timing is reported only after the backends agree on the result.

2.2 The four functions

Function	SQL backend	File backend	Unit
Scan	`count(*) WHERE title LIKE '%kw%'`	`title.contains(kw)`, one streaming pass	1 pass
Point	`SELECT ... WHERE id = ?`, prepared	binary search on a sorted file by offset index	2,000 keys
Update	`replace(name,'T','Ttt')`	`String.replace`, then rewrite the file	2M rows
Counter	`UPDATE ... SET v = v + 1`	read-modify-write `counter.txt` + optional `fsync`	N × 500

Table 2: One SQL statement per operation; the full SQL is in Appendix A.

The four functions cover the costs that separate a DBMS from a file: sequential reading (scan), index lookup (point), extra writes and space reuse (update), and concurrent correctness plus durability (counter).

Two design choices affect fairness. The SQL point batch issues 2,000 separate single-row queries, which is what a client pays to read one row; a set-based `id = ANY` query would avoid that cost and was not tested. The file index is a sorted copy plus 8-byte offsets, built offline and not maintained during writes; the B-tree is. Without its index, the file finds the same keys in one pass.

2.3 How the runs are grouped

Group	What changes	What it tests
Scan	Table size (four scales) and keyword selectivity (four keywords)	Whether selectivity changes a full scan; a leading wildcard cannot use the primary-key B-tree
Point	Index on or off; four table sizes; 2,000 indexed keys	Index cost against the fixed cost of starting a query
Update	Storage engine (PostgreSQL heap, ASTORE, or USTORE); a declared length limit	Space growth, space recovery, and rule enforcement
Counter	1, 4, 16, or 64 clients; file `fsync` on or off; both databases keep `fsync` on	Lost updates and the price of durability

Table 3: The four experiment groups.

Comparisons run at matched parallelism: PostgreSQL uses no parallel workers in its one-core series and four in the four-worker series, openGauss uses one thread by default (`query_dop=1`) and four in the parallel test, and the file backend is always single-threaded. Rows with four workers therefore compare the two databases, not the database with the file.

3 Implementation

3.1 Architecture

`Main` parses the options and runs the repetitions, `Semantics` holds the single definition of each operation, and `Backend` is implemented by `SqlBackend` and `FileBackend`. Sharing one definition keeps the three sides comparable: a change cannot silently apply to one backend only.

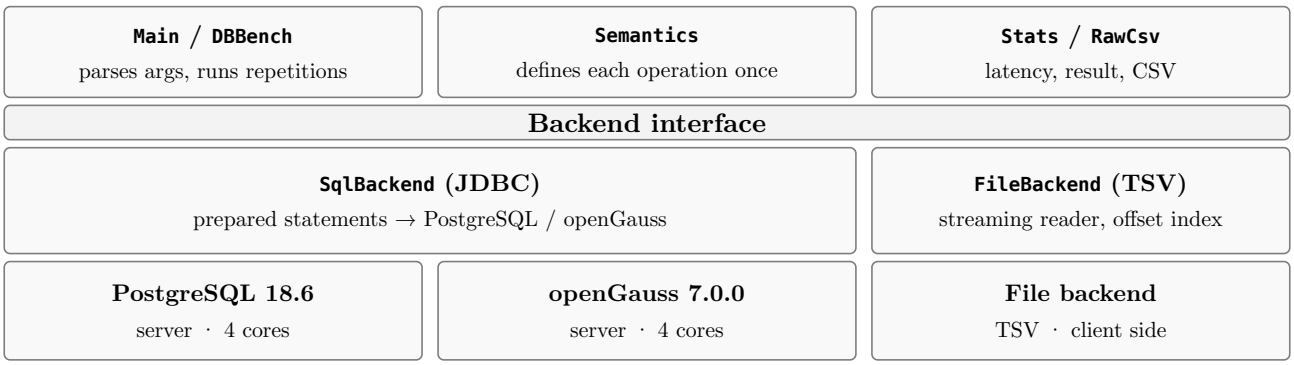


Figure 1: The benchmark components.

3.2 SQL backends

`SqlBackend` uses JDBC prepared statements and no ORM. A scan is a `count(*)` with a `LIKE` parameter, point lookups reuse one prepared single-row `SELECT`, the update is one statement with `replace`, and the counter is a single `UPDATE`. Connection setup is outside the timed region; statement execution and row transfer are inside it. Each repetition starts from a restored table state, so a case measures one operation, not the leftover space from the run before.

3.3 File backend

`FileBackend` has three physical layouts: a sequential scan, a sorted copy plus a fixed-width offset index for point lookups, and a rewrite of the whole file for updates. Updates and counter writes can call `FileChannel.force` to pay the same durability cost as a commit. The backend is single-threaded.

3.4 Measurement steps

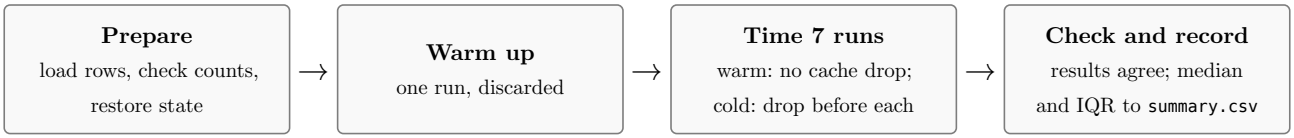


Figure 2: One measured case.

Cold cases drop the OS page cache before every repetition and leave `shared_buffers` intact, so the cold number is not a true cold start. Reported values are medians of seven runs; the IQRs are in the data files.

4 Results and Evaluation

4.1 Full scan: bytes and row checks

Titles	file (ms)	PG 1 core (ms)	OG 1 core (ms)
12.8M	649	866	1317
25.7M	1317	1637	2950
51.4M	2681	3345	5904
102.7M	5693	6778	11685

Table 4: Warm full scan for `the`, median of 7.

On one core the file wins at every size. At 102.7M rows it needs 5.69 s against 6.78 s for PostgreSQL and 11.69 s for openGauss. Two things explain the gap: the file reads 3.58 GB of TSV where the heap is 6.64 GB, and it performs no visibility check on each row.

Selectivity changes the numbers, not the ranking. Across keywords matching 0.05% to 50% of rows, database times move by 10 to 13%, and every engine still reads every row because a leading wildcard cannot use a B-tree (Table 12). Cold runs are closer: with the OS page cache dropped, PostgreSQL is 6% faster than the file at 102.7M (6.93 s against 7.39 s), because `shared_buffers` still holds part of the heap. Figure 3 shows both series; the lines are parallel, so the gap stays about the same.

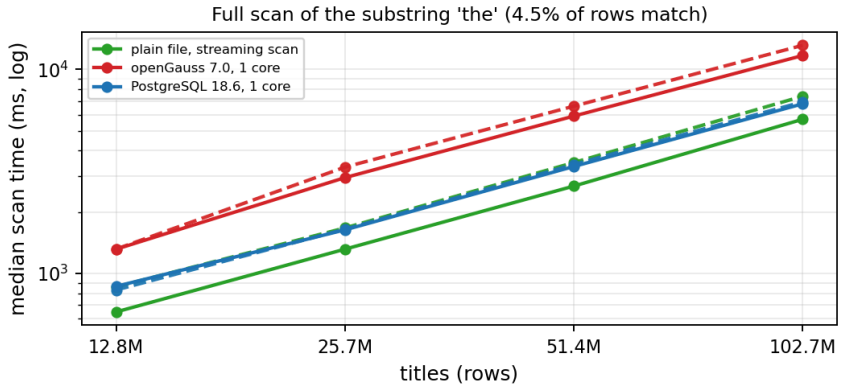


Figure 3: Scan time against size; solid is warm, dashed is cold.

4.2 Point lookups: a fixed cost per query

Titles	file index (ms)	PG B-tree (ms)	OG B-tree (ms)
12.8M	18.5	50.9	80.5
25.7M	23.0	49.3	84.0
51.4M	25.7	51.3	82.1
102.7M	25.4	50.4	81.6

Table 5: Warm point lookups by key, median of 7.

The file's offset index answers 2,000 keys in 25.4 ms, PostgreSQL in 50.4 ms, and openGauss in 81.6 ms. All three take about the same time at every table size, for different reasons. The file performs one binary search per key over a compact offset array. The databases pay a fixed cost for protocol, parsing, planning, and execution on every query; at this scale that cost is the whole cost.

The file index is smaller (8.0 bytes/row against 28.2 for the PostgreSQL B-tree) because it stores offsets into a sorted copy rather than keys and heap pointers. Remove the index and the result reverses: 10

unindexed lookups cost PostgreSQL 10.4 s and openGauss 66.9 s (Table 13). A set-based `id = ANY` query would reduce the per-query overhead and was not tested, so the file's advantage applies only to single-row queries from the client.

4.3 Growing update: space and latency

Engine	Growth	After VACUUM FULL	UPDATE (ms)
PG heap	6.79%	153.6 MB	610
ASTORE	8.36%	167.1 MB	796
USTORE	5.97%	141.6 MB	1395

Table 6: `T → Ttt` on 2M names; sizes include the index.

The rewrite grows every touched row, the worst case for an append-only heap. PostgreSQL's heap grows 6.79%, openGauss ASTORE 8.36%, and USTORE 5.97%. USTORE achieves the smaller relation by keeping the live row in place and moving the old version to undo space, which `pg_relation_size` does not count. It also has the highest update latency: 1,395 ms against 610 ms for PostgreSQL and 796 ms for ASTORE. The space saving is real, but it comes with that latency.

`VACUUM FULL` restores PostgreSQL and USTORE to within about 0.2% of their starting size; ASTORE does not return to its starting size. The file backend rewrites the 2M-row file in 151.7 ms without fsync, faster than any database, but it rewrites everything, blocks writers, cannot roll back, and checks no constraint. Table 15 and Figure 6 in Appendix C give the full storage table and the VACUUM time.

4.4 Constraints: the database can refuse

The same rewrite on a table with `VARCHAR(30)` and a matching `CHECK` shows what a constraint adds. At every limit tested, both databases refuse the statement and roll back with zero rows changed. The file backend writes the violating rows and reports nothing: at `n = 10` it creates 11,815 violations on top of 193,444 already-invalid rows; at `n = 30` it still creates 21. A file program can check a limit if it is written to, but the check is not part of the data model and does not cover every writer. Table 16 in Appendix C lists every limit.

4.5 Concurrency and durability

Clients	file survived	file+fsync survived	PG survived	OG survived
1	500/500	500/500	500/500	500/500
4	8/2,000	6/2,000	2,000/2,000	2,000/2,000
16	2/8,000	1/8,000	8,000/8,000	8,000/8,000
64	1/32,000	7/32,000	32,000/32,000	32,000/32,000

Table 7: Counter with N clients, 500 increments each.

The file backend is fastest at one client because it never asks the disk to confirm the write. Add one fsync per update and it takes 0.52 s, the same as PostgreSQL.

However, at 64 clients the file backend keeps only 1 of all 32,000 increments (7 with fsync), while both databases keep all 32,000. No amount of buffering or retrying fixes this and the correctness needs a lock or a transaction manager.

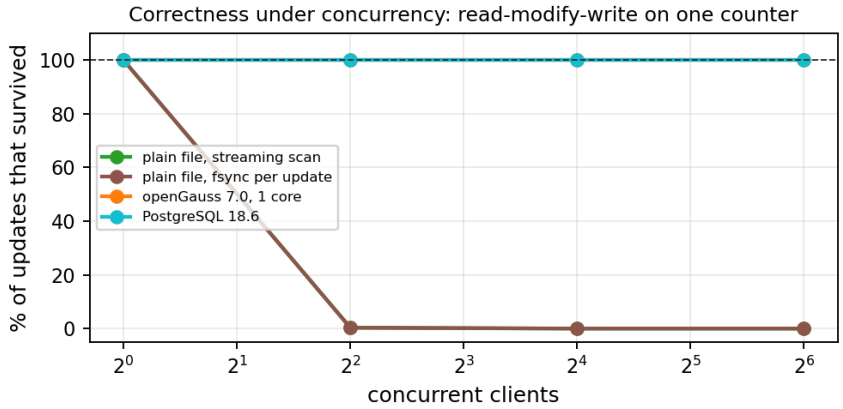


Figure 4: Share of updates that survived.

4.6 Which database is better

Dimension	PostgreSQL	openGauss	Winner
Single-core scan, warm	6,778 ms	11,685 ms	PG, 1.72x
Four-way scan, warm	2,061 ms	3,126 ms	PG, 1.52x
Indexed point lookups (2,000 keys)	50.4 ms	81.6 ms	PG, 1.62x
Durability, 64 clients / 32,000 commits	33.07 s	64.76 s	PG, 1.96x
UPDATE latency, 161,542 rows	610 ms	796 ms (ASTORE) 1395 ms (USTORE)	PG
Space after a growing UPDATE	6.79%	5.97% (USTORE) 8.36% (ASTORE)	USTORE

Table 8: Comparison at 102.7M rows unless noted.

PostgreSQL wins every database-to-database row: 1.72x on one core, 1.52x at four workers, 1.62x on indexed lookups, and about 2x on committed durability. Its B-tree is also smaller (28.2 against 35.6 bytes per row). openGauss's only win is the relation size after the growing update, and that comparison excludes USTORE's undo. At equal parallelism PostgreSQL is the better default for this workload; USTORE is worth considering only when space after a growing update is what matters most. The full comparison is Table 17 in Appendix C.

4.7 Limits

- Single machine and one dataset. Absolute times do not carry over to other machines; the ratios do.
- The 102.7M-row table is eight copies of the title base, so the title distribution repeats.
- The file backend is single-threaded Java on the client cores. A parallel, C, or `mmap` version was not tested; it would likely narrow the scan gap.
- Cold runs retain `shared_buffers`. There is no crash-recovery test, so durability here means fsync on commit.
- The counter uses one row. No multi-row transactions, isolation levels, or read/write mixes were tested.
- Measurements are medians of seven runs; no formal significance test was run.

5 Conclusion

On a warm full scan and on indexed point lookups, a plain file can beat both databases in this setup: it reads fewer bytes, pays no visibility cost on each row, and pays no per-query overhead. That advantage disappears once the work needs concurrent increments, rollback, constraints, or recovery. PostgreSQL is the stronger general-purpose engine at equal parallelism; openGauss's USTORE is a space trade-off with higher write latency, not a throughput alternative. The numbers read like a price list: the file is cheaper because it does not buy the guarantees.

References

1. PostgreSQL 18 documentation. postgresql.org/docs/18.
2. openGauss documentation. docs.opengauss.org.
3. IMDb datasets. datasets.imdbws.com.
4. PostgreSQL JDBC driver. jdbc.postgresql.org.
5. openGauss JDBC driver. gitee.com/opengauss/openGauss-connector-jdbc.

Appendix A: SQL used in the experiments

The full scripts, including the data-loading and cache-control helpers, are in `sql/`.

A.1 Schema

```
CREATE TABLE bench_titles_active (
    id    BIGINT PRIMARY KEY,
    title TEXT   NOT NULL
);

CREATE TABLE bench_people_active (
    pid        BIGINT PRIMARY KEY,
    first_name TEXT,
    surname    TEXT
);

CREATE TABLE bench_counter (
    id INT PRIMARY KEY,
    v  BIGINT NOT NULL
);
INSERT INTO bench_counter VALUES (0, 0);
```

A.2 Retrieval

```
-- scan: one statement, keyword bound as a parameter
SELECT count(*) FROM bench_titles_active WHERE title LIKE '%the%';

-- point: one prepared statement, executed 2,000 times
SELECT id FROM bench_titles_active WHERE id = ?;
```

A.3 Update

```
UPDATE bench_people_active
   SET first_name = replace(first_name, 'T', 'Ttt'),
       surname    = replace(surname,    'T', 'Ttt')
 WHERE coalesce(first_name,'') LIKE '%T%'
    OR coalesce(surname,'')    LIKE '%T%';
```

A.4 Constraint check

```
CREATE TABLE bench_people_strict (
    pid        BIGINT PRIMARY KEY,
    first_name VARCHAR(30),
    surname    VARCHAR(30),
    CONSTRAINT strict_name_len CHECK (
        length(coalesce(first_name,'')) <= 30 AND
        length(coalesce(surname,''))    <= 30
    )
);

-- the UPDATE above is then refused and rolled back by both engines
```

A.5 Storage engines and sizes

```
CREATE TABLE bench_people_astore (
    pid        BIGINT PRIMARY KEY,
    first_name TEXT,
    surname    TEXT
) WITH (storage_type = ASTORE);
```

```
CREATE TABLE bench_people_ustore (
    pid        BIGINT PRIMARY KEY,
    first_name TEXT,
    surname    TEXT
) WITH (storage_type = USTORE);

SELECT pg_relation_size('bench_titles_active') AS heap,
       pg_indexes_size('bench_titles_active')  AS index;
```

A.6 Data checks

```
-- all of these must return 0
SELECT count(*) FROM bench_titles_active
 WHERE title LIKE '%' || chr(9) || '%';
SELECT count(*) FROM bench_titles_active
 WHERE title LIKE '%' || chr(13) || '%'
    OR title LIKE '%' || chr(10) || '%';

-- LIKE and substring must agree on the same 200,000 titles
SELECT count(*) FROM (
  SELECT title FROM bench_titles_active
   WHERE title LIKE '%\%%' OR title LIKE '%\_%'
   ORDER BY id
   LIMIT 200000
) s
WHERE (title LIKE '%the%') <> (position('the' in title) > 0);
```

Appendix B: Configuration

The server container is pinned to physical cores `0,2,4,6` and the client to `8,10`, with no hyperthread sharing between them. The two engine containers never run at the same time. Cold runs write `3` to `/proc/sys/vm/drop_caches` before each repetition and leave `shared_buffers` intact.

Parameter	PostgreSQL 18	openGauss 7.0	Note
shared_buffers	4 GB	4 GB	identical
work_mem	64 MB	64 MB	identical
maintenance_work_mem	1 GB	1 GB	identical
max_connections	200	200	identical
wal_level	replica	hot_standby	equivalent
synchronous_commit	on	on	identical
fsync	on	on	identical
full_page_writes	on	on	identical
checkpoint_timeout	15 min	15 min	identical
random_page_cost	1.1	1.1	identical
autovacuum	off	off	identical
jit	off	-	no counterpart
max_parallel_workers_per_gather	4	-	no counterpart
effective_cache_size	10 GB	10 GB	identical
enable_thread_pool	-	off	no counterpart
io_method	worker	-	no counterpart
enable_ustore	-	off	no counterpart
sql_compatibility	-	PG	no counterpart

Table 9: Server parameters.

PostgreSQL ran with JIT off and `max_parallel_workers_per_gather` at 0 or 4; openGauss ran with `query_dop` at 1 or 4 and chose ASTORE or USTORE per table. Both engines keep `fsync` and `synchronous_commit` on.

Appendix C: Full measurement tables

C.1 Parallel scan

Configuration	PostgreSQL	openGauss	OG / PG
1 worker / thread, warm	6834 ms	11687 ms	1.71x
4 workers / threads, warm	2061 ms	3126 ms	1.52x

Table 10: Warm full scan at 102.7M rows.

C.2 Cold cache

Titles	file	OG 1 core	PG 1 core	method
12.8M	861	1319	831	drop_caches
25.7M	1673	3322	1650	drop_caches
51.4M	3497	6602	3423	drop_caches
102.7M	7386	13114	6929	drop_caches

Table 11: Cold full scan; `shared_buffers` is not cleared.

C.3 Selectivity

Keyword	Rows matched	Hit rate	file (ms)	PG 1 core (ms)	OG 1 core (ms)
`a`	51,890,856	50.51%	4566	5984	11610
`the`	4,609,768	4.49%	5693	6778	11685
`man`	1,004,200	0.98%	5151	6142	10667
`love`	50,888	0.05%	5462	6361	11032

Table 12: Warm scan time by keyword at 102.7M rows, median of 7.

C.4 Point lookups, including the unindexed case

Titles	file index (2,000, ms)	PG B-tree (2,000, ms)	OG B-tree (2,000, ms)	file, no idx (2,000, 1 pass, ms)	PG, no idx (10, ms)	OG, no idx (10, ms)
12.8M	18.5	50.9	80.5	551	1099	6789
25.7M	23.0	49.3	84.0	1109	2058	16688
51.4M	25.7	51.3	82.1	2246	4172	33558
102.7M	25.4	50.4	81.6	4726	10436	66926

Table 13: Warm point lookups; SQL no-index columns use 10 keys.

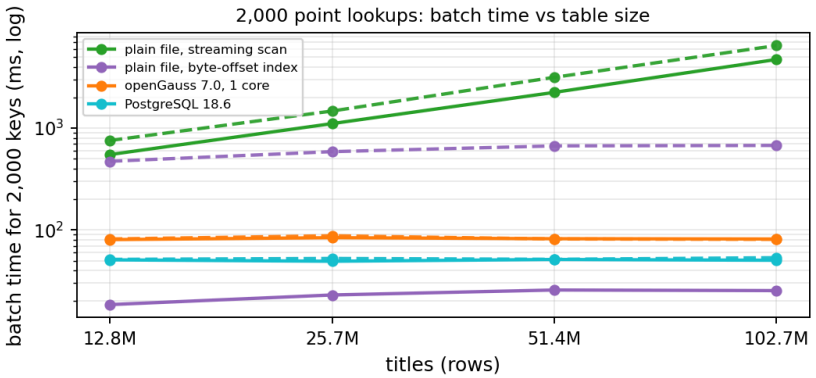


Figure 5: Indexed point lookups against size.

C.5 Storage and engine details

Layout	Bytes/row	Total	Index bytes/row
plain file (TSV)	34.8	3.58 GB	n/a
file + byte-offset index	42.8	4.40 GB	8.0
PostgreSQL heap	64.7	6.64 GB	n/a
PostgreSQL heap + B-tree	92.9	9.54 GB	28.2
openGauss heap	64.8	6.66 GB	n/a
openGauss heap + B-tree	100.4	10.31 GB	35.6

Table 14: Storage per row at 102.7M rows, index included.

Engine	Rows	Before (MB)	Growth	Dead tuples	VACUUM FULL (ms)	After FULL (MB)	UPDATE (ms)
PG heap	161,542	153.3	6.79%	161,009	772	153.6	610
ASTORE	161,542	154.0	8.36%	161,542	1393	167.1	796
USTORE	161,542	141.3	5.97%	161,542	1169	141.6	1395

Table 15: Growing update on 2M names; autovacuum is off.

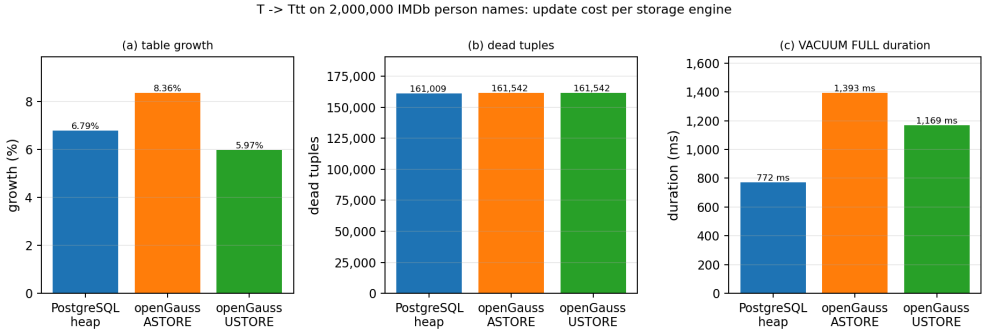


Figure 6: Update cost: growth, dead tuples, and `VACUUM FULL` time.

C.6 Constraint checks

Limit n	New violations	Already invalid (file)	DB rows	PostgreSQL	openGauss	file backend
10	11,815	193,444	1,806,556	refused	refused	11,815 written
15	2,441	27,594	1,972,406	refused	refused	2,441 written
20	344	3,973	1,996,027	refused	refused	344 written
25	92	814	1,999,186	refused	refused	92 written
30	21	248	1,999,752	refused	refused	21 written

Table 16: Constraint test; the file keeps every row.

C.7 Full comparison

Dimension	PostgreSQL	openGauss	Winner
Single-core scan, warm	6,778 ms	11,685 ms	PG, 1.72x faster
Four-way scan, warm	2,061 ms	3,126 ms	PG, 1.52x faster
Cold scan, single core	+2.2%	+12.2%	PG
Indexed point lookups (2,000 keys)	50.4 ms	81.6 ms	PG, 1.62x faster
Unindexed point lookups (10 keys)	10,436 ms	66,926 ms	PG, 6.41x faster
Durability, 1 client / 500 commits	0.52 s	1.03 s	PG, 1.99x faster
Durability, 64 clients / 32,000 commits	33.07 s	64.76 s	PG, 1.96x faster
UPDATE latency, 161,542 rows	610 ms	796 ms (ASTORE) 1395 ms (USTORE)	PG
Space after a growing UPDATE	6.79%	5.97% (USTORE) 8.36% (ASTORE)	USTORE
VACUUM FULL, final size	153.6 MB	141.6 MB (USTORE) 167.1 MB (ASTORE)	USTORE
B-tree bytes per row	28.2	35.6	PG

Table 17: Comparison by dimension

USTORE saves space only in the relation: its undo is outside `pg_relation_size`; its update latency is 1,395 ms against 610 ms.

Appendix D: Source and reproducibility

Path	Contents
`DBBench.java`	Single-file submission build of the benchmark
`benchmark/src/bench/`	Modular benchmark code: `Main`, `Semantics`, `SqlBackend`, `FileBackend`
`sql/`	Schema, data checks, constraint, storage-engine, and size scripts
`scripts/`	Load, duplicate, export, run, restore, and cache scripts
`analysis/`, `report/gen_tables.py`	Consistency checks, figures, and generated tables
`out/raw.csv`, `out/summary.csv`	Per-run measurements and the medians used here

Table 18: Where the code and the measured data live.

```
# compile and run one case
javac -cp "lib/*" DBBench.java
java -cp ".:lib/*" DBBench --backend file --mode scan \
     --keyword the --data ./data

# regenerate the tables and the PDF
python3 report/gen_tables.py
typst compile --root . report/report.typ report/report.pdf
```